

SEDICT: Secure, Energy-sensitive, and Distributed Cluster Training Protocol for Emerging IoT Applications

Jean Frédéric Myoupo^d, Alain Bertrand Bomgni^{b,c}, Hafiz Munsub Ali^e, Yann B. Mtopi Chebu^b, Etienne Gnimpieba Zohim^a

^aUniversity of South Dakota, 4800 N Career Avenue, 57107, SD, USA

^bDepartment of Mathematics and computer science, University of Dschang, P.O. Box. 67, Dschang, Cameroon

^cDepartment of Math, Science and Technology, Oglala Lakota College, 490 Piya Wiconi Rd, Kyle, 57752, SD, USA

^dModelisation, Information and Systems Lab, University of Picardie-Jules Verne, 33 rue Saint Leu, Chemin du Thil 80025 Amiens Cedex 1, Amiens, France

^eComputer Information Systems School of Computing, Binghamton University, 4400 Vestal Pkwy E, 13902, NY, USA

Abstract

The proliferation of heterogeneous Internet of Things (IoT) devices presents unique challenges for network management, particularly in **clustering** where devices exhibit significant disparities in resource capacities, notably memory. Traditional **clustering** protocols, often designed for homogeneous wireless sensor networks, fail when memory-constrained IoT nodes cannot fully participate, leading to incomplete network partitioning and suboptimal performance. This paper introduces **SEDICT** (Secure, Energy-sensitive, and Distributed Cluster Training), a novel protocol developed to support inclusive and resilient cluster formation within heterogeneous IoT environments. **SEDICT**'s core innovation lies in its distributed virtual memory allocation mechanism, which allows memory-rich nodes to temporarily lend storage to memory-poor nodes, ensuring that all devices can engage in the **clustering** process. The protocol unfolds in three phases: neighbor discovery, dynamic virtual memory configuration, and a resilient multi-step **clustering** algorithm that forms mutually exclusive cliques while incorporating security measures against malicious nodes. The simulation results demonstrate that **SEDICT** successfully integrates all nodes into the **clustering** process, a significant improvement over schemes that do not address memory disparity. Although incurring a manageable energy overhead for virtual memory management, **SEDICT** guarantees comprehensive network participation, crucial for the reliability and effectiveness of emerging IoT applications.

Keywords: Internet of Things (IoT), **Clustering**, Distributed Memory, Virtual Memory, Heterogeneous Networks, Network Security, Energy-aware Protocols

Email addresses: jean-frederic.myoupo@u-picardie.fr (Jean Frédéric Myoupo), abomgni@olc.edu (Alain Bertrand Bomgni), hali@binghamton.edu (Hafiz Munsub Ali), yannchebu@gmail.com (Yann B. Mtopi Chebu), etienne.gnimpieba@usd.edu (Etienne Gnimpieba Zohim)

1. Introduction

The proliferation of Internet of Things (IoT) devices is transforming numerous domains, building upon foundations laid by wireless sensor networks (WSNs). A WSN is typically an ad hoc network of small, communicating sensor objects deployed to collect information autonomously [1, 2]. Once deployed, these networks must operate without direct human intervention [3, 4]. However, the often resource-constrained nature of these sensors, particularly their limited memory and processing capabilities, necessitates lightweight protocols. To optimize resource utilization, techniques such as network partitioning into clusters (or cliques) are employed, where nodes within a clique can typically communicate with neighbors in a single hop [5, 6]. Many such partitioning protocols rely on nodes exchanging messages to form these cliques.

A critical challenge arises because most existing protocols assume all nodes possess sufficient memory to execute the protocol and store all necessary state information, such as messages from neighbors. This assumption is often invalid in practice, especially within heterogeneous IoT ecosystems where device capabilities vary significantly. When network nodes lack adequate memory, they may be unable to participate effectively in **clustering**, leading to their exclusion or even protocol failure. To overcome this, mechanisms for creating and allocating virtual memory to resource-scarce nodes are essential, enabling them to execute protocols without impediment [7].

This paper specifically addresses the challenge of network partitioning within heterogeneous IoT environments characterized by disparities in node memory. The focus is on scenarios where some nodes lack sufficient memory to run standard **clustering** protocols. To this end, we introduce **SEDICT** (Secure, Energy-sensitive, and Distributed Cluster Training protocol). The core innovation of **SEDICT** is a novel mechanism for configuring and allocating distributed virtual memory, thereby ensuring that even memory-constrained nodes can fully participate in the **clustering** process. We then apply this virtual memory framework within a robust **clustering** algorithm, adapting and significantly extending foundational secure **clustering** concepts, such as those explored by Sun et al. [8], to specifically accommodate nodes with insufficient memory resources. The proposed protocol ensures that the entire network can be effectively and inclusively organized. The main contributions of this work are:

- A novel distributed virtual memory allocation and management mechanism specifically designed to enable full participation of memory-constrained nodes in IoT **clustering**.
- The **SEDICT protocol**, an inclusive, secure, and distributed **clustering** solution that partitions heterogeneous IoT networks into mutually exclusive cliques while dynamically addressing memory disparities.
- A comprehensive evaluation demonstrating **SEDICT**'s ability to ensure complete node inclusion in the **clustering** process, a critical advantage over protocols that do not explicitly manage memory limitations.

- An analysis of the performance trade-offs, particularly the energy implications of implementing the virtual memory system to achieve comprehensive network participation.

The remainder of this paper is organized as follows: Section 2 presents various works dealing with clustering protocols. Section 3 describes our contribution in detail, including the SEDICT protocol. Section 4 deals with the experimental setup and discusses the simulation results. Finally, Section 5 concludes the paper and outlines potential avenues for future research.

2. Related works

Clustering protocols are used to appoint a self-organizing architecture in a network in order to facilitate operations such as data collection, and routing [9, 10, 11, 12]. The clustering protocols are generally compared using metrics such as sensors energy, distribution, scalability of the network, time and complexity [13, 14, 15, 16, 17, 18, 19, 20]. In this section we present some recent works on clustering. In [8], K. Sun et al. proposed a secure and distributed Cluster-First (CF) clustering protocol. By exchanging information with their neighbors, the normal nodes are divided into several mutually disjoint groups (also known as cliques) within which all nodes can communicate with each other in one hop. Abdul at al. [21] proposed a protocol named P-LEACH that improves network lifetime. The energy consumption is less than the one by LEACH [22] and the protocol decreases the number of dead nodes. Abdul et al. [23] presented APTEEN, a protocol that combines both reactive and proactive schemes by varying the count time (Maximum value of time period among 2 sequential data reports send by a particular member node) and as well as threshold values energy consumption can be made controllable.

In [9], Choaming et al. proposed a multi-hop routing protocol that minimizes network load and consumed energy in order to increase data transfer efficiency in the network. Cluster-heads (CHs) are chose based on the remaining energy of the nodes. The intra-cluster path is computed by the transmission of the entire routing table toward different CH. The results of this method show that it disperses the network load. A protocol named IOT-LEACH is presented by Trupti et al. in [10]. That protocol works with a random selection of CH at each round. CHs provide messages in order to cluster the network and at each round, a CH changes. If a CH has not used all of its energy during its turn, it may still have enough energy to continue as a CH in the next round, otherwise, it can no longer be CH. IoT LEACH solves this limit by defining a threshold in the existing LEACH protocol so that if a CH ends its round still having more energy than the defined threshold, then it can continue to the next round. This will control the energy of the network and therefore minimize it. However, it only authorizes one-hop communications.

Recently, the emergence of technologies such as LoRa has aroused the interest of many researchers around the world. The authors in [19] proposed a clustering algorithm based on LoRa technology. This algorithm divides the network into layers and in each layer, an intelligent gateway selection process is applied. This algorithm is compared to other protocols in the literature and proves that it is very energy efficient and with an interesting packet

delivery rate. However, it still presents the problem of formation time of multi-hop layers, which can be improved.

All the previously presented **clustering** protocols assume that the sensors are homogeneous and that they have enough memory to store all the information exchanged during the **clustering** protocol. In IoT, network can be heterogeneous and thus made up of nodes with different storage memories; this would mean that some nodes might not have enough memory to execute the **clustering** protocol. In these cases, it is necessary to find mechanisms to fill the memory gap and thus allow the execution of the protocol. In that perspective, we propose a solution allowing a node in lack of memory to create virtual nodes in which some information related to the **clustering** protocol can be stored.

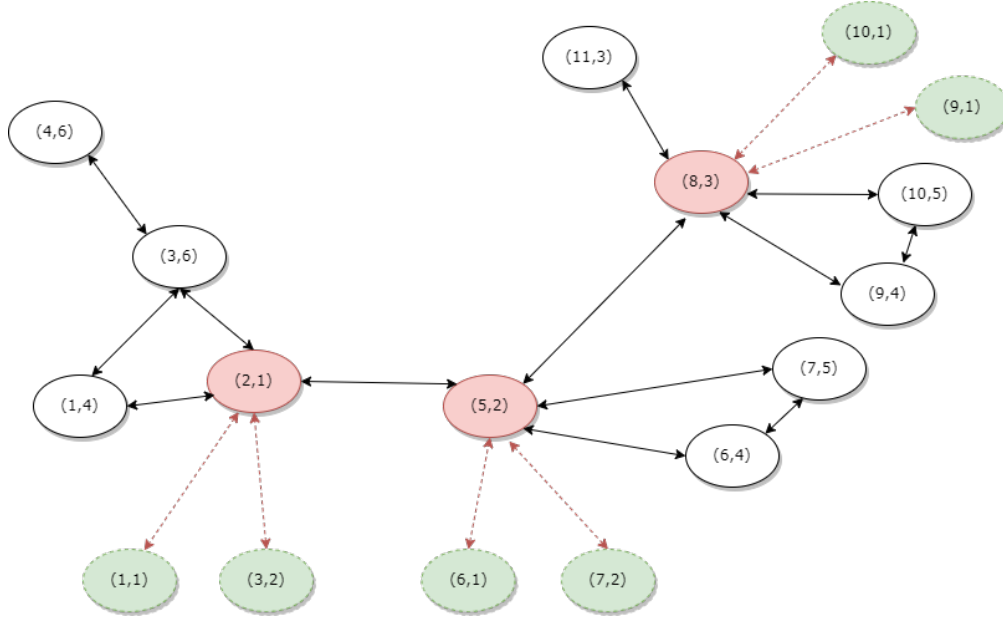


Figure 1: Protocol Overview

For example, Figure 1 presents a network consisting of physical nodes numbered from 1 to 11, where 3 nodes are in lack of memory to run the **clustering** protocol (nodes 2, 5 and 8) and 6 nodes offer virtual memory to those in lack of memory (nodes 1, 3, 6, 7, 9 and 10). Each node is identified by a pair (i, j) where i is the node identifier and j is the node memory. The node colored in green are virtual ones. The virtualization principle that we propose in this paper allow node 2 to create two virtual nodes $[(1,1)$ and $(3,2)]$ in physical nodes 1 and 3 respectively, node 5 to create two virtual nodes $[(6,1)$ and $(7,2)]$ in physical nodes 6 and 7 respectively, and node 8 to create two virtual nodes $[(9,1)$ and $(10,1)]$ in physical nodes 9 and 10 respectively.

3. Protocol Design and Mechanisms

In this paper, we are interested in **clustering** protocol in IoT networks with the particularity that some network nodes may not have enough memory to store information from

Notation	Description
i	Index of node i
L_i	Neighbor list of node i
IN_i	Virtual storage node for node i
M	Required memory to run the protocol
M_i	Memory of node i
M_{mi}	Additional memory required by node i
M_{ri}	Remaining memory of node i
tab_{pi}	Page table of node i
Q	List of nodes having a remaining memory
G_i	List of node i 's neighbors needing additional memory
S_i	List of node for which i acts as virtual node

Table 1: Some notations

their neighbors. Thus, we propose the implementation and management of a distributed shared memory allowing these nodes to store some information and retrieve them when there is a need. To achieve this, we firstly define some key concepts, secondly, we describe how we configure the distributed shared memory, and finally, we apply this configuration on the clustering protocol proposed by Sun et al[8].

The following assumptions are made for the proposed clustering scheme:

- Objects, also known as sensors or nodes are identified by their ID , $1 \leq ID \leq n$, where n is the number of nodes in the network
- At the deployment, each node knows the ID of all the others

Table 1 shows some annotations used to better describe our protocol.

3.1. Neighbours discovering

Before the clustering process starts, each node must know its neighbouring nodes also as their memory storage size. Indeed, that information will allow nodes that are in a need of virtual memory to use the neighbours' memory if possible. The process of neighbours' discovering is described as follows: a broadcast is done by all the nodes after their deployment. That broadcast starts from the node with the smaller ID to the node with the higher ID. The said broadcast is a message containing the node ID and its memory size. During this phase, when a node is broadcasting, all the others are listening. After $\psi_1 = n$ slots, all the nodes know the IDs of their neighbours and their corresponding memory size, and from there, the clustering process is launched in Section 3.2.

3.2. Distributed shared memory configuration

In this section, we describe how we configure the distributed shared memory to allow nodes with memory shortages to store some information in the memory of nodes with surplus memory, while ensuring that the remaining memory on each node is always positive. For that

end, we proceed as follows: firstly, we present how to choose storage nodes and secondly, we manage the conflicts on the virtual memory between different nodes in the network.

3.2.1. Context and system model

Consider an $\text{IoT}(n; M_i)$, $1 \leq i \leq n$, having n elements, with element i having its local memory of capacity M_i . Let $M_{\min}$ be the minimum of M_i . Let p be a node having $M_{\min}$ as the size of its local memory. If p has several neighbors, it is possible that $M_{\min}$ is not sufficient to store all the data resulting from communications with its neighbours. To overcome this problem, we implement a distributed shared memory in which node p can store additional information that it cannot store in its local memory. It is important to note that the virtual memory visible by a node is a function of its one-hop neighborhood. All the nodes in the network are made up of :

- **A handler:** it looks for requested pages from the other network's nodes. When accessing a page in the network in read or write mode the node addresses itself to the handler.
- **A mapper:** which is responsible for answering the page requests from other nodes in the network.

The mapper and handler consist of a page table containing information about the pages of a node. This information is stored in a tab_{pi} table which contains the following fields:

- Access: represents the last access mode to the page: read, write, nil. By default, it is nil.
- Site: the node where the page is kept.
- Lock: acts as a semaphore variable for mutual exclusion access to the page, two primitives are used:
 - LOCK() to test the semaphore and possibly block the requester;
 - UNLOCK() to unlock access.
- Owner : indicates the owner of the page.
- Page number: corresponds to the page number exactly as in paginated memories.

3.2.2. Reading and writing on the network

Reading and writing on the network must take into account two different cases: the case of a node in lack of memory and the case of a node with surplus memory.

1. Case of a node in lack of memory

A node in lack of memory must mainly talk with its Handler either in writing (to find a site for storing its information), or in reading (to retrieve the information it needs). Thus, it executes algorithms 3.1 and 3.2 respectively in read and write mode on the network.

Algorithm 3.1: Handler in reading mode

```
1 Begin
2   LOCK( $tab_{pi}[P].lock$ ) ;
3    $tab_{pi}[P].access = read$  ;
4   UNLOCK( $tab_{pi}[P].lock$ ) ;
```

Algorithm 3.2: Handler in writing mode

```
1 Begin
2   LOCK( $tab_{pi}[P].lock$ ) ;
3    $tab_{pi}[P].access = write$  ;
4   Send( $p$  to  $tab_{pi}[P].site$ ) ;
5   UNLOCK( $tab_{pi}[P].lock$ ) ;
```

2. Case of a node with remaining memory

For a node with remaining memory, the other nodes will mainly address its mapper in write or read mode to save and retrieve certain information. Algorithms 3.3 and 3.4 describe the different processes.

Algorithm 3.3: Mapper in reading mode

```
1 Begin
2   LOCK( $tab_{pi}[P].lock$ ) ;
3    $tab_{pi}[P].access = read$  ;
4   Send( $p$  to requesting node  $tab_{pi}[P].owner$ ) ;
5   UNLOCK( $tab_{pi}[P].lock$ ) ;
```

3.2.3. Choice of storage nodes

The objective here is firstly to allow nodes in lack of memory to choose among their neighbors, the nodes on which they will store their information, and secondly, to allow nodes with enough remaining memory, to satisfy those in need of memory. We propose Algorithm 3.5 in that purpose.

To allow nodes know their neighbors, a broadcast is done by all the nodes as stated in Section 3.1. From this point, each node stores the list of its neighbors and is able to know the amount of memory it needs for the whole clustering protocol execution. We assume that Algorithm 3.5 is executed by node i . Initially, if i needs a virtual memory (Line 4) and it has not yet received a notification from a neighbor that accepts to acts as virtual node for it (Line 5), i sends a request to its first neighbor which is in the list of nodes that have remaining memory (Q) in the network (Line 9). After i receives a notification from a neighbor (Line 5),

Algorithm 3.4: Mapper in reading mode

```
1 Begin
2   LOCK( $tab_{pi}[P].lock$ ) ;
3    $tab_{pi}[P].access = write$  ;
4   Save( $P$ ) ;
5   UNLOCK( $tab_{pi}[P].lock$ ) ;
```

it firstly adds the node from which it receives the notification in its storage node (Line 6), then it updates its additional memory required (Line 7) before executing Line 8 in order to make sure it still needs additional memory in case the previous updating did not satisfy all its requirements. At this level, node i knows that it will store w items in the virtual memory located to node j (The notification sender).

If i does not need additional memory to store data during the routing process, then, it may have remaining memory that can serve as virtual memory. In this later case, if it receives a request to host data from node j (Line 11), it firstly checks if its remaining memory is able to store all the data of node j (Line 12). If this is the case, then, node i updates its remaining memory by reducing the amount of memory needed by node j (Line 13), following by the sending of the acceptance notification to node j (Lines 14 and 16). In the case where the requested memory was greater or equal than the remaining memory of node i , it removes itself from the list of nodes able to act as virtual node (Line 17). In Line 18, node i adds node j in its list of nodes for which it will serve as virtual node.

3.2.4. Conflict management

During the virtual memory choosing procedure, it may happen that a node having remaining resource (Let's say node i) receives many requests from nodes in need of virtual memory. In that case, the request processing is done in the first-in first-out (FIFO) approach, meaning that node i will not be able to send a notification to node j before a notification is sent to the node whose request was received by i before node j 's request. In the other way, a node in need of virtual memory (Let say node j) will cancel the request it has send to node i after a given period if it has not received any notification from i . In that case, j will send a new request to its next neighbor which is present in the list of nodes having remaining memory (Q). Finally, it should be noted that access to list Q is protected such that many objects cannot write within it at the same moment, but many access for reading is possible. Furthermore, no reading is possible when there is a write operation on the list.

3.2.5. Filling the page table

At the end of each computation, the mappers synchronize the different page tables so that each node has complete information about all the places where it needs to store its data. Indeed, since that during the virtual memory attribution process all the nodes were saving information about the nodes that will serve as virtual memory (Line 6) and for which they will act as virtual node (Line 18), they can fill the page table at the end of Algorithm

Algorithm 3.5: Virtual memory attribution

```

Input:  $L_i$  /* i's neighbors */
1  $Q = \{j | 1 \leq i \leq n, M_{ri} > 0\}$  ranked in ascending order
2  $G_i = \{j \in L_i | M_{mj} > 0 \text{ and } j \neq i\}$  ranked in ascending order
3 Begin
4   if  $M_{mi} > 0$  then
      /* Meaning that the actual node is in lack of memory and needs a
      virtual one */
5     if Notification( $i, w$ ) is received from node j then
6        $IN_i \leftarrow IN_i + j$ ;
7        $M_{mi} \leftarrow M_{mi} - w$ ;
8     if  $M_{mi} > 0$  then
9       Send Request( $Q[k]$ );
      /* Where  $k$  is the ID of first neighbor of  $i$  in  $Q$  */
10    else
11      if ( $M_{ri} > 0$  and Receive Request( $j$ ) from node j) then
          /* Meaning that the current node has a remaining memory that
          can serve as virtual memory */
          /* Where  $j$  is the next node in  $G_i$  */
12      if ( $M_{ri} - M_{mj} > 0$ ) then
13         $M_{ri} \leftarrow M_{ri} - M_{mj}$ ;
14        Send Notification( $j, M_{mj}$ );
15      else
16        Send Notification( $j, M_{mj} - M_{ri}$ );
17         $Q \leftarrow Q - i$ ;
18       $S_i \leftarrow S_i + j$ ;
```

3.5 execution.

3.3. *Clustering protocol*

In this section, we exploit the memory management method presented in section 3.2 and the principles of network virtualization to propose a *clustering* protocol for heterogeneous IoT and wireless sensor networks. To be able to execute the cluster formation protocol, some conditions must be fulfilled :

- The sum of the remaining memory must be greater or equal than the sum of additional memory required by the nodes in lack of memory:

$$\sum M_{ri} \geq \sum M_{mi}$$

- For any node i of the network, the sum of the list sizes of the neighborhood of i must be less than or equal to the sum of the remaining memory of the neighborhood.

The *clustering* protocol obeys the following vocabulary: C_i denote the clique to which node i belongs. A node is normal if it follows the entire process of cliques formation, otherwise it is a malicious node. Normal node should have consistent cliques as defined by clique conformity. Indeed, there is clique conformity for a normal node i if whatever the node $j \in C_i$, $C_j = C_i$, in other words, if there is a node $j \in C_i$ such that $C_j \notin C_i$, it is said that there is an inconsistency of clique. It is assumed that each node has a unique ID, that it can be uniquely identified. In addition, a message sent by a normal node will be well received in time by all its neighbors. The presented protocol takes place in five steps if there is no click inconsistency and in six steps otherwise.

3.3.1. *Configuration of distributed shared virtual memories and creation of virtual nodes*

Let (i, M_i) denotes node i with its local memory M_i . Nodes are subdivided into two categories:

- Virtual nodes: their size is less or equal than those of their parents. Furthermore, they contain a subset of information that will be processed by their parents, and they do not process any information since they just serve as data storage for their parent node.
- Physical nodes: They are responsible of data processing for virtual nodes for which they act as parents. To this end, they keep information about the cliques obtained during the clique formation process.

Let a node i of capacity M_i with q_i neighbors, three situations may happen:

1. if $M_i < q_i$, then node i computes $M_{mi} = q_i - M_i + 1$. Meaning that i will need M_{mi} storage memory to follow the *clustering* protocol.
2. if $M_i > q_i$, then i computes $M_{ri} = M_i - q_i - 1$. Meaning that i will have M_{ri} available storage memory and then, it will be able to act as virtual node for its neighbors.

i	1	2	3	4	5	6	7	8	9	10	11
M_i	4	1	6	6	2	4	5	3	4	5	3
q_i	2	3	3	1	4	2	2	4	2	2	1
M_{mi}		3			3			2			
M_{ri}	1		2	4		1	2		1	2	1
V_i	(2, 1)		(2, 2)			(5, 1)	(5, 2)		(8, 1)	(8, 1)	

Table 2: Shared memory configuration

3. if $M_i = q_i$, the node does not need an additional memory to complete the clustering protocol and it is not also able to act as virtual node for its neighbors.

From this point, node i knows if it may act as virtual node for the neighbors or not. It then executes Algorithm 3.5 in that regard. For example, considering the network shown in Figure 1 that is populated by 11 nodes described as follows (1,4), (2,1), (3,6), (4,6), (5,2), (6,4), (7,5), (8,3), (9,4), (10,5), (11,3), and as explained above, node 2 can't store all the information of its neighbors; so it will compute $M_{m2} = 3 - 1 + 1 = 3$. In the other case node 3 will compute $M_{r3} = 6 - 3 - 1 = 2$ and node 1 will compute $M_{r1} = 4 - 2 - 1 = 1$. At this point nodes 3 will execute Algorithm 3.5 to end up with node 2 using nodes 1 and 3 as virtual nodes. Table 2 presents the shared memory configuration obtained from the example of Figure 1 where i represents the nodes' ID and $V_i = (l, m)$ means node i will act as virtual node for node l with m items.

3.3.2. Computation of the local maximum clique (LMC)

Since that each node i has in its memory a neighbor list L_i that stores the IDs of its one-hop neighbors, in this second step, all nodes exchange their neighbor lists with all their neighbors. This is done from the node with the smaller ID to the node with the higher ID and that process requires $\psi_2 = n$ slots. For nodes with virtual nodes, data processing will be done in turn for each virtual node. The objective here is to determine the largest group (also called Local Maximum Clique or LMC) to which a node can belong. To start, each sensor i sends the list L_i of its neighbors to all the others. Upon reception, each node i updates the matrix M that represents its connections with its one-hop neighbors. This is done using Equation 1:

$$M[i, j] = \begin{cases} 1 & \text{if } j \in L_i \\ 0 & \text{if } j \notin L_i \end{cases} \quad (1)$$

Table 3, presents the matrix M that records the connections of all the node between themselves from the network presented in Figure 1 (i represents the nodes' ID):

In matrix M , unidirectional links are considered as non-existent links. With this matrix of neighbors, each node calculates the LMC to which it belongs. Indeed, based on the node i neighbors, graph $G_i = (V_i, E_i)$ is built, where V_i is made up of node i and its neighbors and E_i is the set of bidirectional links between the nodes in V_i . The LMC calculation is done using Algorithm 6 that runs in rounds.

i	1	2	3	4	5	6	7	8	9	10	11
1	1	1	1	0	0	0	0	0	0	0	0
2	1	1	1	0	1	0	0	0	0	0	0
3	1	1	1	1	0	0	0	0	0	0	0
4	0	0	1	1	0	0	0	0	0	0	0
5	0	1	0	0	1	1	1	1	0	0	0
6	0	0	0	0	1	1	1	0	0	0	0
7	0	0	0	0	1	1	1	0	0	0	0
8	0	0	0	0	1	0	0	1	1	1	1
9	0	0	0	0	0	0	0	1	1	1	0
10	0	0	0	0	0	0	0	1	1	1	0
11	0	0	0	0	0	0	0	1	0	0	1

Table 3: Neighbors Matrix

i	1	2	3	4	5	6	7	8	9	10	11
C_i^2	{1, 2, 3}	{1, 2, 3}	{1, 2, 3}	{3, 4}	{5, 6, 7}	{5, 6, 7}	{5, 6, 7}	{8, 9, 10}	{8, 9, 10}	{8, 9, 10}	{8, 11}
C_i^3	{1, 2, 3}	{1, 2, 3}	{1, 2, 3}	{3, 4}	{5, 6, 7}	{5, 6, 7}	{5, 6, 7}	{8, 9, 10}	{8, 9, 10}	{8, 9, 10}	{8, 11}
C_i^4	{1, 2, 3}	{1, 2, 3}	{1, 2, 3}	{4}	{5, 6, 7}	{5, 6, 7}	{5, 6, 7}	{8, 9, 10}	{8, 9, 10}	{8, 9, 10}	{11}
C_i^5	{1, 2, 3}	{1, 2, 3}	{1, 2, 3}	{4}	{5, 6, 7}	{5, 6, 7}	{5, 6, 7}	{8, 9, 10}	{8, 9, 10}	{8, 9, 10}	{11}

Table 4: Obtained LMC after executing Algorithm 4 in the network of Figure 1

Algorithm 3.6: Determination of the Local Maximum Clique (LMC)

Input: $G_i = \{V_i, E_i\}, i \in V_i$

Output: C_i

- 1 $S_i = \{j | (i, j) \in E_i\};$
 - 2 $C_i = \{i\};$
 - 3 $L_i \leftarrow \phi$
 - 4 **while** $S_i \neq \emptyset$ **do**
 - 5 find $k \in S_i$ with the biggest $|L_i \cap L_k|$
 - 6 $L_i \leftarrow L_i \cup L_k$
 - 7 $C_i \leftarrow C_i \cup \{k\}$
 - 8 $S_i \leftarrow S_i - \{k\} - \{j | (j, k) \notin E_i, j \in S_i\}$
-

L_i contains neighbors of i which are likely to be in the same clique as i . In every round, node i chooses a neighbor and adds it to its LMC C_i . Node i maintains the set S_i containing the list of its neighbors that can be selected in the next round. Initially, all neighbors of i are in S_i and C_i contains only i . In the first round, node i calculates the number of links it has in common with each of its neighbors, then it chooses the neighbor k with whom it has the maximum common neighbors number $|L_i \cap L_k|$, after which node i removes k from S_i and puts it in C_i . Node i also removes all node from S_i that are not directly connected to k . In the second round, node i searches in S_i for the sensor which has the maximum of neighbors in common with the nodes in C_i (i.e. nodes i and k). node i removes it from S_i and puts it in C_i , thereafter it removes all the nodes of S_i that are not directly connected with the last node put in C_i . Node i repeats the same process until S_i is empty. At the end of the algorithm, node i sorts the node of C_i with respect to their ID and obtains its LMC C_i^1 (C_i^k designates the clique of node i at the end of step $k, 2 \leq k \leq 5$). This step does not required information exchanged since that nodes use information obtained from the neighbors discovering to locally compute the LMC. The second line of Table 4 shows the LMC computed from the network presented in Figure 1 after the execution of Algorithm 6.

3.3.3. Order and update the LMC

The LMC calculated in Section 3.3.2 may be different depending on the node. In this third step, each node consults the LMC of its neighbors to update its own. In order to prepare its final conforming clique, each node i sends C_i^2 to all its neighbors. For security reason, each message is authenticated by $\mu TESLA$ [24]. As node i computes C_i^2 by a heuristic based on information from its neighbors, it is possible that i receives more LMC C_j^2 from a neighbor j than it does contains. Hence, after receiving these LMCs from its neighbors, node i checks if

there is a click C_j^2 better than C_i^2 . To compare the clicks calculated with the different node, the following relation is defined $\overset{i}{\prec}$ between clicks as follows: $C_j \overset{i}{\prec} C_k$ if and only if

1. $i \in C_j, i \in C_k$ and
2. (a) $|C_j| < |C_k|$, or
(b) $|C_j| = |C_k|$, but $c_j < c_k$, where $c_j = \min\{a_i | a_i \in C_j \wedge a_i \notin C_k\}$ et $c_k = \min\{b_i | b_i \in C_k \wedge b_i \notin C_j\}$ or
(c) $c_j = c_k$ but $j < k$

$C_j \overset{i}{\prec} C_k$ if the number of node in C_k is greater than the number of node in C_j ; or if they contain the same number of node but the *ID* of the first two different node are such that $j < k$; or cliques C_j and C_k are identical but $j < k$. Suppose i receives n cliques containing it, it orders them so as to have $C_{\alpha_1}^2 \overset{i}{\prec} C_{\alpha_2}^2 \overset{i}{\prec} \dots \overset{i}{\prec} C_{\alpha_i}^2 \overset{i}{\prec} \dots \overset{i}{\prec} C_{\alpha_n}^2$ and therefore it updates its clique with $C_{\alpha_n}^2$ and we get $C_i^3 = C_{\alpha_n}^2$. This step takes $\psi_3 = n$ slots to complete since each node sends C_i^2 once. Line 3 of Table 4 shows the updated and ordered LMC obtained at this step from the LMC computed in Section 3.3.2.

3.3.4. Getting the final clique

In this step, each node i sends its updated clique C_i^3 to its neighbors. As in step 3 of this **clustering** protocol, each message sent in this fourth step is authenticated by the $\mu TESLA$ protocol. For each node $j \in C_i^3$, node i checks if it belongs to C_j^3 ; if not, it removes j from C_i^3 , after that it gets its final clique C_i^4 . If node i does not receive the updated clique from one of its neighbors j , it simply removes j from its clique. This step is performed in $\psi_4 = n$ slots (Time require for each node to send C_i^3 to its neighbors).

Still considering the running example, Line 4 of Table 4 shows the result obtained in this stage. Here it is noted that node 4 removes node 3 from its neighbors list since that node 4 does not appears in the node 3 neighbors list. In the same way, node 11 removes node 8 from its neighbors list. After these first 4 steps, the conformity of the cliques is guaranteed if all the sensors are normal. However, in hostile environments, compromised nodes can participate in the protocol, hence the need for the extra steps to detect clique inconsistency and malicious nodes appears.

3.3.5. Verification of the cliques conformity

The clique conformity verification is the fifth step. Each node i also calculates a security code based on the messages received in the previous steps, signs it, and adds it to the message containing the final clique. When a normal node receives the first copy of the final clique C_j^4 from its neighbor j or sent by another neighbor, if $j \in C_i^4$, then j resend the clique C_j^4 . The purpose of this resend is to prevent silent attacks¹. Each node i checks the conformity of the clique, i.e. node i checks that for all $j \in C_i^4$, $C_j^4 = C_i^4$. If a clique inconsistency is detected, node i enters step 6, otherwise it ends the process of forming the cliques. This step is performed in $\psi_4 = n$ slots.

¹There is a silent attack by a malicious node when the latter sends messages to some of its neighbors and not to others

3.3.6. Identification of Malicious nodes or Clique Inconformity

This sixth step is carried out in two stages: the cliques conformity verification and the removal of malicious nodes. The conformity verification aims to identify the malicious nodes that sent inconsistent messages in the previous steps. When malicious nodes are detected by node i , the latter sends an alert to the other nodes using the signature of the malicious node as proof. After removing the malicious nodes from the network, the protocol is restarted. This stage is done as follows: if a node i detects an inconsistency of a clique with node j , node i asks node j to send it all the messages it has received from the previous steps. As node j received the final authenticated clique C_i^4 at step 4, only if $C_i^4 \neq C_j^4$, node j provides all of its previous messages received from i . Node j needs to sign these messages to prove that they were sent by it. For an effective signature, node j must calculate a security code in relation to the messages then, sign and send this code in a single message. After checking the signature of node j , if node i finds that j correctly followed the first five steps of the protocol, i re-checks the first four steps of the protocol for j :

- If the resulting final clique of j is not the same as that received by i from j at step 5, then node j is a malicious node and i uses the signature of j to inform others;
- Otherwise, i checks if for all the nodes $k \in L_i \cap L_j$, k sent it the same message as in j . If node k at one step sent two different messages to i and j , node i uses the conflicting messages as proof to inform its other neighbors that k is a malicious node;

After identifying the malicious nodes, node i enters the second stage.

The second stage is as follows: when a silent attack is launched by a malicious node, a normal node may not have the evidence to convince other normal nodes that have received messages from the malicious node. In addition, the normal node cannot distinguish the normal node that has actually detected a malicious node from the malicious node that seeks to trigger a false alarm on a normal node. In each of these two cases, the protocol ensures that all normal nodes complete the clique conformance by performing the following actions: if two normal nodes i and j seek inconsistency, that is, $j \in C_i^4, i \in C_j^4$ and $C_i^4 \neq C_j^4$, except loss of generalities, it exists $k \in C_i^4$ and $k \in C_j^4$, then:

1. If $k \in C_i^3, k \notin C_j^3$, node i has received C_k^2 and node j has not received it, then node i removes j from C_i^4 and node j removes i from C_j^4 .
2. if $k \in C_i^3 \cap C_j^3$, node j has received C_k^3 and $j \notin C_k^3$, node i does not received C_k^3 , then i removes k from C_i^4 .

As stated by Sun et al.[8], the advantages of the clustering protocol presented in this paper are twofold: (1) it ensures that at the end of the clustering any node is in a well defined clique, (2) it is secure. To these advantages, we have added the virtual memory management to make the IoT protocol more realistic as possible.

Theorem 1. *The proposed secure distributed cluster training (SEDICT) protocol in IoT networks takes place in $O(n)$ time slots if we do not consider internal processing made by network' nodes, where n is the number of nodes in the network.*

Proof. Indeed, the total time slot required for all the communications during the whole clustering process is given in Equation 2 which corresponds to $O(n)$ in the Lando notation

$$\psi = \psi_1 + \psi_2 + \psi_3 + \psi_4 + \psi_5 = n + n + n + n + n\psi = 5n \quad (2)$$

□

4. Simulations and result

SEDICT protocol has been evaluated in this section using performance metrics like energy consumption (during virtual memory configuration and during the clustering phase), average number of cliques, average number of nodes per clique and average number of messages exchanged during virtual memory configuration. Simulations have been performed on OMNET++ simulator installed on a core i5 Laptop with 16 GB of RAM and Windows 10 operating system. Simulation parameters are presented in table 5.

Parameter	Value
Number of sensors	100-500
Transmission range	20m; 40m; 60m; 80m
Deployment area	100m*100m
Remaining memory	20; 40; 60; 80
Amplification energy (e_{amp})	0.5J
Energy mitigation (N)	$2 \leq N \leq 4$
Transmission energy (e_t)	3J
Reception energy (e_r)	2J

Table 5: Simulation parameters

4.1. Virtual memory configuration evaluation

The virtual memory configuration is evaluated in Figure 2. To that end, simulations are done considering the total memory that network nodes needs ($\sum M_{mi}$) to properly accomplish the clustering. In Figure 2a, when $\sum M_{mi}$ varies from 20 to 80, we notice that the number of exchanged messages for virtual memory also increases, meaning that the more a network will need virtual memory, the more the number of exchanged messages for virtual memory configuration will be. Furthermore, in Figure 2b the energy consumption of nodes when the virtual memory configuration is done increases when $\sum M_{mi}$ increases from 20 to 80. The later observation goes in the same line as the observation made for Figure 2a. We then conclude that the energy consumption of nodes increases when the number of exchanged message does so.

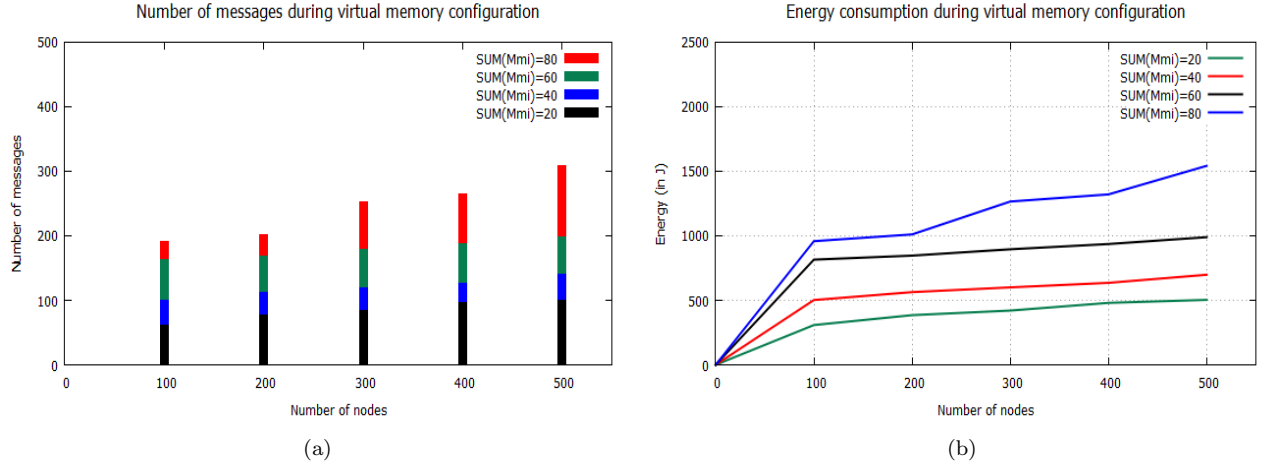


Figure 2: Memory configuration analysis

4.2. Evolution of cliques and nodes per clique

The number of cliques as well as the number of nodes per clique were evaluated in our simulations as shown in Figure 3. We then noticed that when the transmission radius varies from 20 meters to 80 meters, the number of cliques decreases (Figure 3a), meaning that the highest is the transmission radius, the less is the number of obtained cliques. Furthermore, Figure 3b shows that when the transmission radius is higher, the number of node per cluster is biggest. This lead us to conclude that after running the proposed **clustering** protocol, the number of node per cluster significantly depends on the transmission radius of network nodes. Moreover, considering Figure 4, it is noted that energy consumption is higher when the transmission radius is high. For instance, for the same value $\sum M_{mi} = 80$, we notice that the energy consumed by the node when the transmission radius is 20 is 4000J for a network populated by 500 nodes (Figure 4a), while the energy consumed by the node when the transmission radius is 80 equals to 6600J when the number of nodes in the network is 500 (Figure 4b). From the previous observations, we conclude that for energy efficiency during the execution of our **clustering** protocol, it is better to fix the nodes transmission radius as lower as possible.

4.3. Comparison of energy consumption between our protocol and the one of Sun et al. [8]

In the final stage of our simulation, we compared our protocol with the one proposed by Sun et al.[8]. As shown in Figure 5, our protocol consumes more energy that the one by Sun et al. This is justify by the fact that Sun et al. did not considered the virtual memory management during their **clustering**, meaning that in the case where a node does not have enough memory resource to participate to the **clustering**, the result may be biased because it will not be able to store and process data from its neighbors.

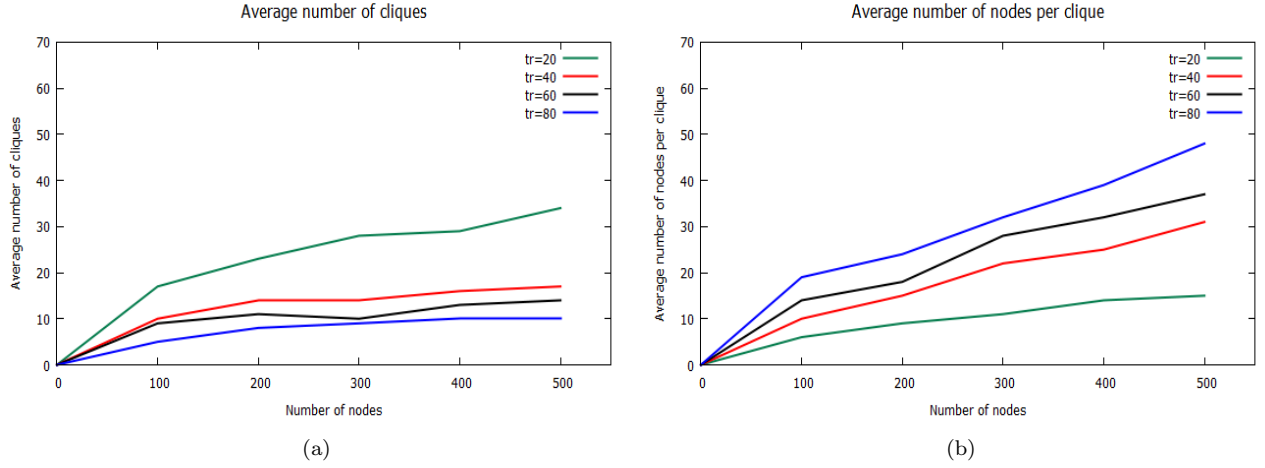


Figure 3: Evolution of cliques and nodes per clique according to the transmission range.

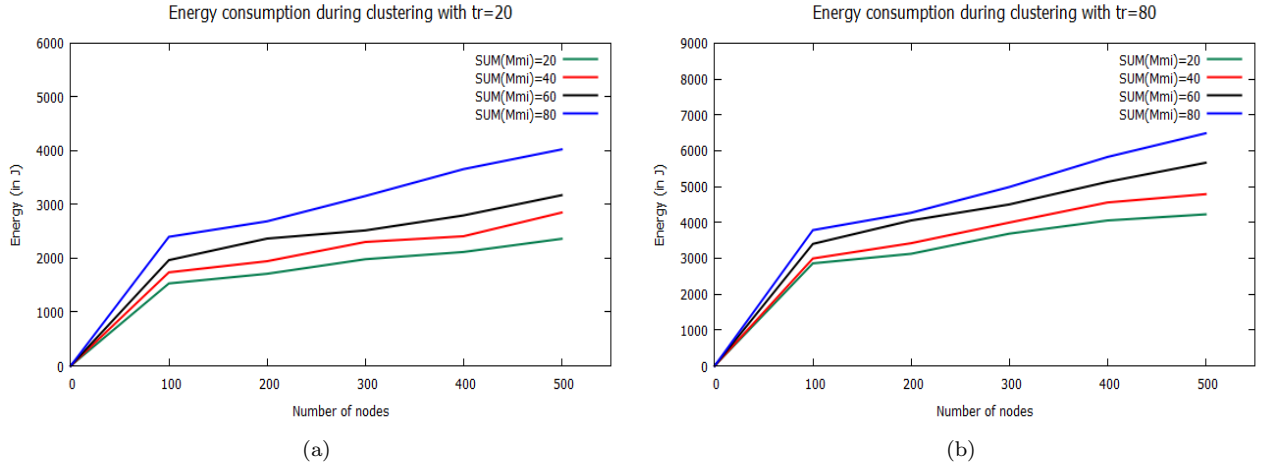


Figure 4: Energy consumption during clustering phase.

5. Conclusion

This paper addressed the significant challenge of memory heterogeneity in forming robust clusters within Internet of Things (IoT) networks, a scenario where nodes may lack sufficient memory to fully participate in standard clustering processes. We introduced SEDICT (Secure, Energy-sensitive, and Distributed Cluster Training protocol), a novel scheme detailed in our contribution section, designed to ensure comprehensive network organization. SEDICT's core mechanism involves configuring distributed shared virtual memories, allowing nodes with surplus memory to act as virtual storage for those with insufficient resources. Following this, the protocol proceeds through a multi-step process: computation of the local maximum clique (LMC), ordering and updating of the LMC, obtaining the final cliques, and verifying clique conformity. An additional step is invoked to handle detected clique inconformities, thereby enhancing the protocol's resilience. The simulation results provide

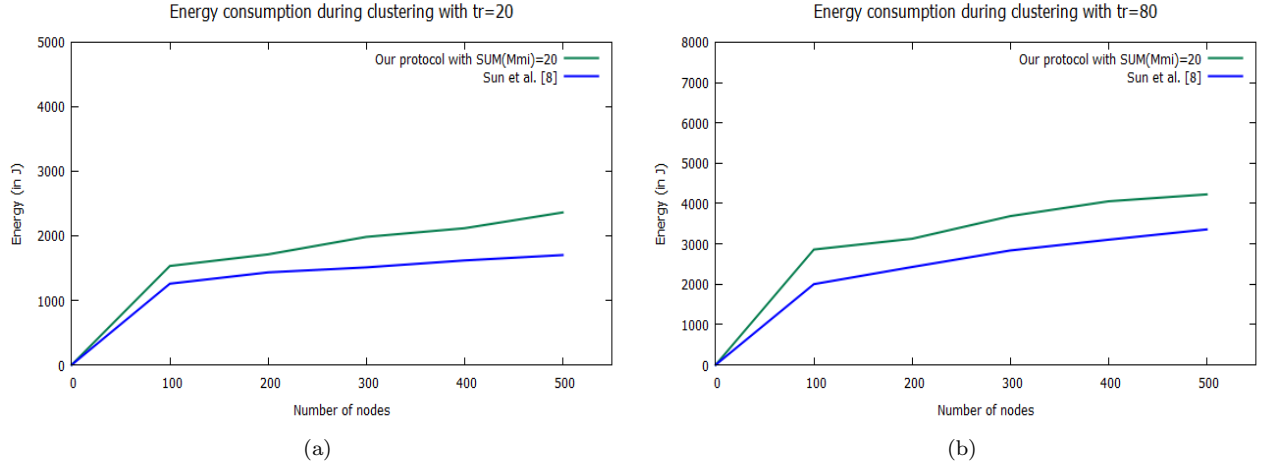


Figure 5: Comparison of energy consumption between our protocol and the one of Sun et al. [8]

compelling evidence for **SEDICT**'s efficacy. We observed that, firstly, the virtual memory configuration phase's energy consumption scales with the amount of virtual memory required for the network to complete **clustering**. Secondly, for improved energy efficiency during **clustering**, configuring network nodes to operate with the lowest feasible transmission range is advisable, though this may affect clique characteristics. Thirdly, and most critically, while **SEDICT** demonstrated higher energy consumption compared to baseline schemes like that of Sun et al. [8], this is a direct consequence of its primary design goal: enabling nodes that initially lack sufficient memory resources to safely and fully participate in network **clustering**. This inclusivity is paramount for the integrity and utility of the resulting network structure. For future work, we aim to further enhance the energy efficiency of **SEDICT**. One promising direction is the integration of advanced link quality prediction techniques, as explored by authors in [25], to optimize communication patterns and reduce energy waste. Furthermore, we intend to explore the application of **SEDICT** principles to **clustering** in multi-hop environments, particularly leveraging Mobile Edge Computing (MEC) architectures [26, 27]. This will involve adapting the virtual memory management and clique formation mechanisms to the dynamic and hierarchical nature of MEC-assisted IoT networks. Ultimately, **SEDICT** offers a robust solution for building well-structured and inclusive clustered IoT networks, laying a stronger foundation for subsequent data aggregation, routing, and application-level services.

6. Acknowledgment

This work was supported by the National Science Foundation [NSF OIA-1849206, OIA-1920954]; and the National Institutes of Health [5P20GM103443-20].

References

- [1] Alain Bertrand Bomgni, Elie Tagne Futé, Miguel Landry Foko Sindjoung, and Clémentin Tayou Djamegni. A tree-based distributed permutation routing protocol in

- multi-hop wireless sensors network. *Wireless Sensor Network*, 8:93–105, 2016.
- [2] Millad Ghane, Sunita Chandrasekaran, and Margaret S. Cheung. Towards a portable hierarchical view of distributed shared memory systems: Challenges and solutions. In *The 11th International Workshop on Programming Models and Applications for Multicores and Manycores (PMAM'20)*, 2020.
 - [3] Alain Bertrand Bomgni, Miguel Landry Foko Sindjoung, Dhalil Kamdem Tchibonsou, Mthulisi Velepini, and Jean Frédéric Myoupo. Neseprin: A new scheme for energy-efficient permutation routing in iot networks. *Computer Networks*, 214:109162, 2022.
 - [4] Bomgni Alain Bertrand, Mdemaya Garrik Brel Jagho, Ali Hafiz Munsub, Zanfack David Gnimpieba, and Zohim Etienne Gnimpieba. Espina: Efficient and secured protocol for emerging iot network applications. *Cluster Computing*, 26(1):85–98, 2022.
 - [5] Miguel Landry Foko Sindjoung, Alain Bertrand Bomgni, Elie Tagne Futé, Clementin Tayou Djamegni, and Gerard Chalhoub. Iscp : An instantaneous and secure clustering protocol for wireless sensor networks. *Network protocols and algorithms*, 10(1):65–82, 2018.
 - [6] Garrik Brel Jagho Mdemaya Alain Bertand Bomgni. An energy-efficient protocol based on semi-random deployment algorithm in wireless sensors networks. *International journal of network security*, 22(4):602–609, 2020.
 - [7] Juhyun Bae, Gong Su, Arun Iyengar, Yanzhao Wu1, and Ling Liu. Efficient orchestration of host and remote shared memory for memory intensive workloads. In *The International Symposium on Memory Systems (MEMSYS 2020)*, 2020.
 - [8] Sun Kun, Peng Pai, and Ning Peng. Secure distributed cluster formation in wireless sensor networks. *22nd annual Computer Security Application Conference, Las Vegas*, pages 131–140, 2006.
 - [9] Chaoming Wang, Yuan Zhang, Xuwen Wang, and Zhiyong Zhang. Hybrid multihop partition-based clustering routing protocol for wsns. *IEEE Sensors Letters*, 2(1):1–4, 2018.
 - [10] Trupti Mayee Behera, Umesh Chandra Samal, and Sushanta Kumar Mohapatra. Energy-efficient modified leach protocol for iot application. *IET Wireless Sensor Systems*, 8(5):223–228, 2018.
 - [11] DL Reddy. A review-efficiency of energy clustering and routing in wireless sensor networks. *Int J Adv Technol*, 10(222):2, 2019.
 - [12] Mohamed Elhoseny, Ahmed Farouk, Nanrun Zhou, Ming-Ming Wang, Soliman Abdalla, and Josep Batle. Dynamic multi-hop clustering in a wireless sensor network: Performance improvement. *Wireless Personal Communications*, 95(4):3733–3753, 2017.

- [13] Oussama Senouci, Zibouda Aliouat, and Saad Harous. Mca-v2i: A multi-hop clustering approach over vehicle-to-internet communication for improving vanets performances. *Future Generation Computer Systems*, 96:309–323, 2019.
- [14] Muhammad Fahad Khan, Farhan Aadil, Muazzam Maqsood, Syed Hashim Raza Bukhari, Maqbool Hussain, and Yunyoung Nam. Moth flame clustering algorithm for internet of vehicle (mfca-iov). *IEEE Access*, 7:11613–11629, 2018.
- [15] Rowayda A Sadek. Hybrid energy aware clustered protocol for iot heterogeneous network. *Future Computing and Informatics Journal*, 3(2):166–177, 2018.
- [16] J Sathish Kumar and Mukesh A Zaveri. Clustering approaches for pragmatic two-layer iot architecture. *Wireless Communications and Mobile Computing*, 2018, 2018.
- [17] Sunil Kumar Singh, Prabhat Kumar, Jyoti Prakash Singh, and Mohammad Abdallah Ali Alryalat. An energy efficient routing using multi-hop intra clustering technique in wsns. In *TENCON 2017-2017 IEEE Region 10 Conference*, pages 381–386. IEEE, IEEE, 2017.
- [18] Piotr Gajewski Wojciech Bednarczyk. An enhanced algorithm for manet clustering based on weighted parameters. *Universal Journal of Communications and Network*, 1(3):88–94, 2013.
- [19] Muhammad Omer Farooq. Clustering-based layering approach for uplink multi-hop communication in lora networks. *IEEE Networking Letters*, 2(3):132–135, 2020.
- [20] Yoonyoung Sung, Sookyoung Lee, and Meejeong Lee. A multi-hop clustering mechanism for scalable iot networks. *Sensors*, 18(4):961, 2018.
- [21] Abdul Razaque, Musbah Abdulgader, Chaitrali Joshi, Fathi Amsaad, and Mrunal Chauhan. P-leach: Energy efficient routing protocol for wireless sensor networks. *Long Island Systems, Applications and Technology Conference (LISAT) IEEE*, pages 1–5, 2016.
- [22] Heinzelman W. and Chandrakasan A. and Balakrishnan H. Energy-efficient communication protocol for wireless microsensor networks. *Proceedings of the 33th IEEE Hawaii International Conference on systems (2000)*, 2000.
- [23] Abdul Rehman K., Nitin Rakesh, Abhay Bansal, and Dev Kumar Chaudhary. Comparative study of wsn protocols(leach, pegasis and teen). *Third International Conference on Image Information Processing, Research Gate, IEEE*, pages 422–427, 2015.
- [24] Adrian Perrig, Robert Szewczyk, Victor Wen, and David Culler and J. D. Tygar. Spins: Security protocols for sensor networks. *Wireless Network*, 8(5):521–534, Sept. 2002.
- [25] Miguel Landry Foko Sindjoung and Pascale Minet. Estimating and predicting link quality in wireless iot networks. *Ann. des Télécommunications*, 77(5-6):253–265, 2022.

- [26] Miguel Landry Foko Sindjoung, Mthulisi Velempini, and Alain Bertrand Bomgni. A mec architecture for a better quality of service in an autonomous vehicular network. *Computer Networks*, 219:109454, 2022.
- [27] Miguel Landry Foko Sindjoung, Mthulisi Velempini, and Clémentin Tayou Djamegni. A data security and privacy scheme for user quality of experience in a mobile edge computing-based network. *Array*, 19:100304, 2023.